

Assignment 1

Learning Outcomes & Materials

This assignment is intended to develop and assess the following unit learning outcomes:

- ✓ **LO1.** Iteratively apply object-oriented design principles to design small to medium-size software systems, using standard software engineering notations, namely UML class diagrams and UML interaction diagrams.
- ✓ **LO2.** Describe the quality of object-oriented software designs, both in terms of meeting user requirements and the effective application of object-oriented design concepts and principles.
- ✓ **LO3.** Apply object-oriented programming constructs, such as abstraction, information hiding, inheritance, and polymorphism, to implement object-oriented designs using a programming language (namely, Java).
- ✓ **LO5.** Apply principles of software engineering practice to create object-oriented systems with peers using tools including integrated development environments (IDEs), UML drawing tools, and version control systems.

To demonstrate your ability, you will be expected to:

- read and understand UML design documentation for an existing Java system
- propose a design for additional functionality for this system
- create UML class diagrams to document your design using a UML drawing tool such as [diagrams.net](#), [UMLet](#) or [plantuml](#) – you are free to choose which one
- write a design rationale evaluating your proposed design and outlining some alternatives
- implement the features of the system that you designed
- use an integrated development environment to do so
- use git to manage your team's files and documents

The marking scheme for this assignment will reflect these expectations

Learning Materials

The base code will be automatically available in your group's repository (Gitlab).

Repeat this mantra: Design, write code, test, fix design, fix code, repeat

⚠ Note: You **must NOT follow** demo apps' design decisions; they only show how to use the engine, **NOT** how to design a proper system with object-oriented principles.

Introduction

For the rest of the semester, you will be working on a relatively large software project. You will design and implement new functionalities to an existing system that we will provide to you.



IMPORTANT: A document explaining the FIT2099 Assignment Rules is available as the [Assignment Survival Kits](#) lesson. Please read it and make sure you understand **BEFORE** you begin the project – you are expected to follow what it says and will almost certainly lose marks if you do not.

In this assignment (Assignment 1), you will be working **individually** to implement the first few features of the game. We highly recommend starting by extracting the whole game features. Then, proceed with designing, testing, and repeating the process as needed. This iterative approach will help ensure a well-developed and refined system.

Getting Started

The initial codebase will be available in a repository that will be created for you on git.infotech.monash.edu. In the meantime, please go through the assignment support modules on Ed Lesson to familiarise yourself with the game engine that you will use during the assignment.



You do not need to submit an interaction diagram (e.g. sequence diagram or communication diagram) in assignment 1. However, you will need to submit these documents for Assignment 2 and Assignment 3. For assignment 1, you may still create these documents if you find them useful for designing the system.

General background

You will be working on a text-based “**rogue-like**” game. Rogue-like games are named after the first such program: a fantasy game named Rogue. They were very popular in the days before home computers were capable of elaborate graphics and still have a small but dedicated following.



If you would like more information about rogue-like games, a good site is <http://www.roguebasin.com/>. The initial codebase will be available in the repository mentioned above. It includes a folder containing design documents for the system.

Designborne

In this assignment, we will be developing the “Designborne” game, inspired by the games such as Dark Souls, Elden Ring and Bloodborne. We may use several similar names (characters, items, locations) and concepts. The purpose of using an actual game’s concepts is to help you visualise the required features, such as watching the video gameplay from the actual game to illustrate features that you may find difficult to comprehend. We also believe that using actual game references can **bring fun** while working on the assignments.

All of the linked game contents, videos, and images belong to the respective owners and are subject to copyright. We mainly use the concepts for educational purposes and provide credit to the original creators accordingly. We may also add, alter, and reduce the original content and features to make them more suitable to the game engine, unit outcomes, and assignments’ time frame.

What’s next?

Below are five slides (REQ1-REQ5) describing the requirements you need to complete. Each requirement includes game features that include background stories, entity descriptions (actors, items, or ground), relationships between entities, and actions between them. We suggest extracting these features into a list and discussing it with the TAs to ensure all features are included (*this discussion is not assessed, but highly recommended*). Following these requirement slides, we will outline the deliverables necessary for the Assignment 1 assessment.

REQ1: The Player

In "Designborne", you play as "The Abstracted One", who has been revived by an unknown entity in the starting location, "The Abandoned Village".

At the beginning of the game, the player has 150 hit points (the health attribute) and 200 stamina. They start the game by waking up in the building in the middle of the map. A building is simply several squares of floor (`. `) and dirt (`. `), surrounded by walls (`#`). The display character of the player is "@".

[illegible]

Figure 1. Example of buildings in the game (red squares). They consist of walls, dirt, and floor grounds. Since the actor cannot walk over the wall, a building may have various openings at various sides (replaced with dirt or floor). For example, the middle building has an opening on the south side (a floor ). The player starts the game in the building in the middle of the map (This image does not contain the player symbol).

The player can use their limbs to attack the enemies, dealing 15 damage with 80% attack accuracy.



Make sure to display the player's hit points and stamina before printing the menu on the console!

In the building, there is a “Broadsword”, which is a starting weapon that the player can pick up and use to attack enemies. This weapon can deal 110 damage with 80% attack accuracy. This weapon is represented by the character “1”. It also has a special skill, “Focus”, that can increase the weapon’s damage multiplier by 10% for 5 turns and set (not increase) the weapon’s hit rate to 90% while the weapon is in the player’s inventory. If the player drops the weapon while the skill is active, the

weapon's damage multiplier and hit rate will revert back to the previous state.



For simplicity, If the player uses the skill again while the skill is still active, you can simply re-apply the damage multiplier increase and reset the turns back to 5. For example, if the weapon skill was activated once, the damage multiplier is up by 10% and assume that the skill has 2 turns left. If the skill is activated at this point, the damage multiplier is up by 20% (increased by another 10%), and the turns would be reset to 5.



You can assume that all weapons have either one skill or none. The stamina points used by activating weapon skills vary between different weapons.

Whenever the player uses “Focus”, which is the Broadsword's weapon skill, their stamina is reduced by 20% of their maximum stamina.

At each turn, the player's stamina will recover by 1% of their maximum stamina.

REQ2: The Abandoned Village's Surroundings

After the player exits the building, they notice that there are several bottomless pits within the village. This bottomless pit is also known as the Void, and it is represented by the display character "+". Any entity, including the player, will be killed if they step on it.



If the player dies, make sure that the game ends and a "YOU DIED" message is displayed on the console (see the FancyMessage class).

The player also notices that there are several graveyards in the village. The graveyard is displayed as "n". As the player gets closer to one of the graveyards, a figure rises up and attacks the player! This is because the graveyard is occupied by the former inhabitants of the village (see REQ3). The graveyard can spawn this enemy with a 25% chance at every turn of the game.



Edit (27/08/2023): For simplicity, you can assume that the graveyard can spawn a WanderingUndead with a 25% chance at every turn, regardless of the player's location.



Hint: if you need to pass in arguments to a ground's constructor, you can do the following

```
gameMap.at(30, 11).setGround(new ConcreteGround(input1, input2));
```

REQ3: The Wandering Undead

The first enemy that the player encounters is the “Wandering Undead”, which is displayed as “t”. If the player is not within the surrounding of the enemy, it will wander around. However, when the player is nearby (within the surrounding of the enemy or one block away from the enemy), it will attack the player. This type of enemy has 100 hit points and can attack the player with its limbs, dealing 30 damage with 50% accuracy. An enemy cannot attack another enemy.

This enemy cannot enter a floor, represented by the display character “_”, so that the player can run back to safety if their health is low. Furthermore, both the player and the enemy cannot enter walls. They, however, can walk around freely in the Void*, on puddles and dirt.

* with consequences (see REQ2)

REQ4: The Burial Ground

After defeating the enemy, the player explores the deeper section of the village until they reach a locked gate (represented by the display character “=”), leading to a new region of the abandoned village, which is known as the burial ground. Since the gate is locked, the player will not be able to proceed without a key.

```
".....+++++++.....~::~~::++.....~",
".....+++++++.....~::~~::+.....~",
".....++++.....~::~~.....++",
".....+.+......~::~.....++",
".....++~::~.....++",
".....++~::~#####.....++",
".....++++~.....#_____#.....++++",
".....+++.....#_____#.....+++++",
".....+++.....###_###.....~::+++++",
".....~::~.....~::~.....++",
".....~::~.....++.....",
".....~::~~::~~::~.....++.....",
".....~::~.....~::~~::~~::~.....~::~.....",
".....+~::~.....+++++++~::~~::~~::~.....",
".....+~::~.....+++++++~::~.....~::~....."
```

A sample burial ground map



The burial ground must be implemented as a new map (separate from the abandoned village map). Make sure to add another Gate in the Burial Ground map to go back to the Abandoned Village map. You can assume that this is a separate gate that the player also needs to unlock.

The wandering Undead has a 25% chance of dropping an “Old Key” (represented by “-”) once they are defeated by the player. For the sake of simplicity, multiple key instances are expected in the game. The player can pick up this key and use it to unlock the gate leading to the burial ground.



In this game, only the player is able to use the gate for travelling (once they have the old key). Other actors may step into the gate once it is unlocked, but they will not be able to use it for travel.



You can assume that the old key is reusable. That is, you don’t need to remove the old key from the player’s inventory after the player uses it.

REQ5: The Inhabitants of The Burial Ground

The graveyards in the burial ground are occupied by a different type of enemy, known as the Hollow Soldier, displayed as "&". This type of enemy has 200 hit points and can attack the player with its limbs, dealing 50 damage with 50% accuracy. The graveyard can spawn this enemy with a 10% chance at every turn of the game.



Similar to the wandering Undead (see REQ4), if the player is not within the surrounding of the enemy, it will wander around. However, when the player is nearby (within the surrounding of the enemy), it will attack the player. An enemy cannot attack another enemy.

This enemy cannot enter a floor, represented by the display character "_", so that the player can run back to safety if their health is low. Furthermore, both the player and the enemy cannot enter walls. They, however, can walk around freely in the Void*, on puddles and dirt.

* with consequences (see REQ2)

Once defeated, this enemy has a 20% chance to drop a healing vial (represented by the display character "a"), which can be consumed by the player to heal 10% of their maximum health. It also has a 30% chance to drop a refreshing flask (represented by the display character "u"), which can be consumed by the player to increase their stamina by 20% of their maximum stamina. Each instance of both healing vials and refreshing flask can be consumed by the player only once.



For simplicity, the calculation of the probability of each item dropping is independent of the other item, so the player can get lucky and get both items after killing one Hollow Soldier.



Healing vials and refreshing flasks cannot be consumed directly from the ground. That is, they must be picked up by the player before being consumed.

The wandering Undead that the player met earlier in the Abandoned Village map (see REQ3), once defeated, also has a 20% chance to drop a healing vial. Similar to the Hollow Soldier, the calculation of the probability of the Healing vial dropping is independent of the Old key.

Submission Instructions



Not following any one of the instructions below will result in penalty being applied to your final submission.



You do not need to submit an interaction diagram (e.g. sequence diagram or communication diagram) in **assignment 1**. However, you will need to submit these documents for assignment 2 and assignment 3. For assignment 1, you may still create these documents if you find them useful for designing the system.



We will mark your assignment based on the latest commit in the `main` branch in your GitLab repository by the due date, **not** in the `master` branch.

Class Diagrams & Documentation Submission

- As mentioned in the Design & Diagrams section, **you MUST create one design diagram per requirement**. In other words, each requirement must be represented in its own separate diagram. Organising them in one large design diagram or combining requirements based on their packages is not allowed. Doing so may reduce the clarity of your design diagram or could be seen as an attempt at reverse engineering. Penalties will be applied if this occurs. Please keep each requirement diagram distinct and avoid any attempts to merge or combine them.
- You **MUST** save your design documentation, including design diagrams, rationale and report (diagram + rationale combined into a single document), in the "docs/design/{assignmentName}" directory. So, for Assignment 1, **you must save all design documentation in the "docs/design/assignment1" directory**.



TAs cannot be expected to look at other directories, except for "docs/design/{assignmentName}", when marking the design diagrams and rationale.

- All design documentation must be saved in PDF format (or PNG/PDF for diagrams, such as REQ1.png, REQ2.png, and so on).
- A Moodle submission is compulsory, and it must be done before the deadline. Please, compress ALL files (code, documentation, and diagrams) as one zip file, and submit the compressed file.



Disclaimer: Although there are multiple ways to design the game, there are also bad designs and good designs - we will mark your submission not against one design but based on the design principles

Design Diagrams

We expect you to produce **five** *UML class diagrams* following **the FIT2099 Assignment Rules**. These Rules are available on [EdLesson](#).

You should not create one class diagram that shows the entire system. The sample diagram in the base code shows the whole system to help you understand how the `game` works with the `engine`. But, in this assignment, you only need to show the following:

- the new parts,
- the parts you expect to change, and
- enough information to let readers know where your new classes fit into the existing system.

As it is likely that the precise names and signatures of methods will be refactored during development, you do not have to put them in these class diagrams. Instead, the overall responsibilities of the classes need to be documented *somewhere*, as you will need this information to begin implementation. This can be done in a separate text document (`.md` markdown format) or spreadsheet, which you should put inside the `docs` directory. We will not assess this document, but we believe it will be handy during the implementation phase.

Design Rationale

 A design rationale should be created for **each** requirement.

To help us understand how your system will work, you must also write a *design rationale* to explain your choices. You must demonstrate how your proposed system will work and **why you chose to do it that way**. Here is where you should write down concepts and theories you've learnt so far (e.g., DRY, coupling and cohesion, etc.). You must consider **the pros and cons** of the design to justify your argument.

The design (which includes *all* the diagrams and text that you create) must clearly show the following:

- what classes will exist in your extended system
- what the roles and responsibilities of any new or significantly modified classes are
- how these classes relate to and interact with the existing system
- how the (existing and new) classes will interact to deliver the required functionality

You are not required to create new documentation for components of the existing system that you *do not* plan to change.

 **IMPORTANT!** We will not accept any Word document because it cannot be opened/displayed in Gitlab.

Implementation

 A reminder: you must not modify the game engine, as stated in the assignment rules document.

Your implementation must adhere to the Google Java coding standards.

 Google Java coding standards: <https://google.github.io/styleguide/javaguide.html#s5-naming>

Write Javadoc comments for *at least* all public and protected methods and attributes in your classes.

You will be marked on your adherence to the standards, Javadoc, and general commenting guidelines that were posted to EdStem earlier in the semester.



To ensure that your work adheres to good coding practices in this unit, we encourage you to minimise the use of `instanceof` and/or [downcasting](#), as they are considered code smells. It's important to note that if there are any instances where you need to use them, please provide appropriate justifications in code comments or design rationale. We believe learning how to properly utilise polymorphism is crucial in addressing this code smell, and we are committed to teaching you how to do so effectively.

Marking Rubric

Assignment 1 rubric

Prerequisite for marking: Design documents (UML diagrams and design rationale) and implementation need to be submitted for the assignment to go through the marking process. Missing any of these will result in 0 marks.

Overview:

Total: 28 points

- Feature implementation completeness (6 points)
- Implementation quality: design principles (8 points)
- Design rationale (4 marks)
- Integration with the existing system (2 points)
- UML syntax and clarity (2 points)
- Alignment and design consistency (2 points)
- Style & Javadoc (1 point)
- Git usage (2 points)
- Format and directory structure (1 point)

Detailed rubric items:

Feature implementation completeness (6 points)

6 marks - The system runs and perfectly meets **all functional expectations** as per the relevant requirement(s) with no runtime errors. All required classes and relationships are implemented, and the program's behaviour aligns perfectly with the specification.

5 marks - The system runs and meets **all functional expectations (with some minor errors or punctual omissions)** as per the relevant requirement(s). All necessary classes and relationships are included, and the program's behaviour largely matches the specification, except for **one or two minor unexpected behaviours**. No runtime errors occur.

4 marks - The system runs and **partially meets all functional expectations, displaying several minor functional errors or omissions** as per the relevant requirement(s). Despite this, the majority of necessary classes and relationships are included, and **only a few minor unexpected behaviours occur**. No runtime errors are present.

3 marks - The system runs and **all functional expectations were addressed to some extent**, (with one or two major functional errors) as per the relevant requirement(s). Most important classes

and relationships are included but **at least one or two major unexpected behaviors** are observed. A major unexpected behaviour is that which affects considerably the completeness of at least one requirement. No runtime errors occur.

2 marks - The system runs but **several functional expectations were not addressed** as per the relevant requirement(s) (e.g. the notion of Behaviour is not included even though it is a part of the requirement). Alternatively, runtime errors occur during the execution of the system but they can be easily fixed.

1 mark - The system runs but **addresses only some functional expectations and shows major omissions** as per the relevant requirement(s) (important classes or relationships are missing). Alternatively, runtime errors occur during the execution of the system without a clear idea of the cause or cannot be easily fixed.

0 marks - The system runs but it **poorly addresses or doesn't address the functional expectations** as per the relevant requirement(s). Alternatively, the system might not run at all.

Implementation quality: design principles (8 points)

This item applies across all functional expectations as per the relevant requirement(s)

8 marks - The implementation of all functional expectations as per the relevant requirement(s) **flawlessly adheres to good design principles and concepts** (e.g., DRY and SOLID principles), making the design easy to extend and maintain. Any punctual violations are **convincingly** justified in the design rationale (e.g., using singleton to implement a feature). All relevant requirements have been addressed.

7 marks - The implementation follows good design principles **nearly perfectly** making the design is easy to extend and maintain (if some punctual principles are violated, the trade-off is **somewhat** justified in the design rationale. All relevant requirements have been addressed.

6 marks - The implementation involves **one or two minor violations of design principles** (could be easily fixed) across all functional expectations as per the relevant requirement(s) (e.g., some attributes are not set to private without any justification). All relevant requirements have been addressed.

5 marks - The implementation involves **minor violations of design principles in multiple places** (could still be easily fixed) across all functional expectations as per the relevant requirement(s). All relevant requirements have been addressed.

4 marks - The implementation involves **one or two non-severe violations of design principles that could be implemented in a better way** (no trade-offs are convincingly provided in the design rationale) across all functional expectations as per the relevant requirement(s) (e.g. some code repetitions are found in the implementation that would require some refactoring to be fixed). All relevant requirements have been addressed.

3 marks - The implementation involves **non-severe violations of design principles in multiple**

places (no trade-offs are convincingly provided in the design rationale). All relevant requirements have been addressed.

2 marks - The implementation involves **one or two severe violations** of design principles that could be implemented in a better way across all functional expectations as per the relevant requirement(s), e.g. violating the basic principles covered so far. Fixing them would require substantial refactoring. Alternatively, not all relevant requirements have been attempted.

1 mark - The design/implementation can be considered **hacky or various instances of procedural programming are found**. For example, the implementation uses downcasting and 'instanceof' in various cases without a convincing justification. Alternatively, it can also be the case that abstraction is not used, making the design **difficult to extend and maintain**. Alternatively, only some relevant requirements have been attempted.

0 marks - The implementation mainly follows a non-OO paradigm; or the UML class diagram(s) or the implementation is missing. Alternatively, most relevant requirements have not been attempted.

Design rationale (4 points)

4 marks - The design rationale includes a description of **what has been done and why**, focusing on the principles and concepts taught in the unit (such as DRY and SOLID principles) and not in terms of game design. The rationale discusses the **advantages and disadvantages** of the design (pros and cons), **the reasons** for choosing the current design, and **ways in which it can be easily extended** (e.g. my design achieves OCP because if a new character is added in the future ...). All relevant requirements have been addressed.

3 marks - The rationale describes **what has been done and why**, based on the principles and concepts taught in the unit. It also includes **some discussion** of the pros and cons of the design and the reasons for the current design choice but **lacks examples of future extensibility**. All relevant requirements have been addressed.

2 marks - The rationale describes **what has been done and why**, aligning with the principles and concepts taught in the unit. However, it **lacks a discussion on the pros and cons of the design and/or examples of future extensibility**. All relevant requirements have been addressed.

1 mark - The rationale **primarily describes what has been done** without a thorough explanation of the decision-making reasons. For instance, it may mention design principles and concepts without providing convincing explanations, discussion of pros and cons, or examples of how the system can be extended. Alternatively, not all relevant requirements have been attempted.

0 marks - The design rationale is either very challenging to read, is missing, or the submitted work omits more than one relevant requirement.

Integration with the existing system (2 points)

This item applies across all relevant requirements.

2 marks - The implementation **effectively uses the engine classes** (e.g. the submitted work demonstrates that the students understand the difference between actions and behaviours). All relevant requirements have been addressed.

1 mark - The implementation **does not use some engine classes as intended** (e.g. behaviours are created for some actions that do not need it, such as actions performed by the player) or includes **custom classes for functionality that could be implemented with the engine class** (e.g. created a custom ground class instead of using the ground class given in the engine package). Most relevant requirements have been addressed.

0 marks - The engine has been modified in a minor or significant way OR the UML class diagram OR the implementation is missing

UML syntax and clarity (2 points)

This item applies across all relevant requirements.

2 marks - Relevant (static and/or dynamic) diagrams are **perfect in terms of syntax**, with no missing multiplicities, correct arrowheads for all relationships, and appropriate usage of realisation for classes implementing interfaces, generalisation for classes or interfaces inheriting others, etc. Diagram formatting is **consistent and clear**. All requested diagrams have been submitted.

1 mark - Diagram(s) contain **some minor syntax errors**, such as missing multiplicities for several associations, inappropriate use of generalisation for classes implementing interfaces, realisation for classes extending others, or classes extending multiple classes, etc. Nonetheless, the design can still be understood by the TA with a little effort. Alternatively, there are several **inconsistencies across diagrams**. All necessary diagrams have been submitted.

0 marks - Diagram(s) are significantly hard to understand or show major inconsistencies in formatting and clarity, OR more than one required diagram is missing, OR they do not resemble a UML diagram as required.

Alignment and design consistency (2 points) - incl. Design rationale, code and UML diagrams

2 marks - The design rationale and UML diagrams are in **perfect alignment** with each other and with the code implementation across all functional expectations, as per the relevant requirement(s). All relevant requirements have been addressed or attempted and they are covered in the implementation, UML diagrams and design rationale.

1 mark - There are **some small inconsistencies** (that can be easily fixed) between the design documents and the implementation, but all relevant requirements have been addressed or attempted, and they are covered in the implementation, UML diagrams and design rationale.

0 marks - There are **major inconsistencies** (which would necessitate significant changes for

correction) or **numerous smaller discrepancies** distributed throughout the design documents and the implementation. Alternatively, one or more than one required diagram or implemented requirement is missing, or some submitted design documents do not resemble the required UML diagram format.

Style & Javadoc (1 point)

1 mark - The code is properly documented with Javadoc across all functional expectations as per the relevant requirement(s), including class-level and method-level documentation. The Google Java Style guide is followed properly (e.g. package names are written in lowercase, attributes and variables names are written in lowerCamelCase, class names are written in UpperCamelCase, etc.). All relevant requirements have been addressed or attempted.

0.5 marks - Some classes and methods are missing Javadoc documentation OR some classes do not follow the Google Java style guide

0 marks - Most classes and methods are missing Javadoc documentation OR The Google Java style guide is not followed.

Git usage (2 points)

2 marks - At least **15 meaningful commits** have been logged (with meaningful comments) each with a descriptive commit comment (note: default comments from the web UI don't count.), ideally, one commit per week previous to the submission deadline.

1 mark - Between **7 and 14 meaningful commits** have been logged (with meaningful comments) each with a descriptive commit comment (note: default comments from the web UI don't count.). OR there are more than 10 commits but ALL were done on the week of the submission deadline.

0 marks - There are less than 7 commits.



IMPORTANT: if most of the code has been (in practice) uploaded in one go the whole assignment will not be marked and 0 marks will be awarded.

Project formatting and directory structure (1 point)

1 mark - If the directory structure suggested in the specification has been followed. All the documents are easily found where expected.

0 marks - The suggested directory structure was not followed and some files may be hard to find.

Handover Interview (Compulsory)

COMPLETED - The student can answer all (or at least 2) questions during the handover interview

satisfactorily. The responses need to demonstrate that the student understands the various parts of the submitted assignment.

NOT COMPLETED If two or more questions are not responded to adequately and sensibly. The remaining question(s) is/are partly responded to, but it is unclear whether the student understands their own work.



IMPORTANT: Failing to have meaningful commits (i.e. showing that the task was progressively completed) and/or failing the handover interview would automatically flag this as a potential case of plagiarism, it will be further investigated using a similarity check software, and **zero marks would be awarded for the full assignment.**

Assignment 1 Q&A Session

Direct link: <https://monash.au.panopto.com/Panopto/Pages/Viewer.aspx?id=5896007a-f924-48e5-a03a-b06f00c16f0c>

Meeting chat:

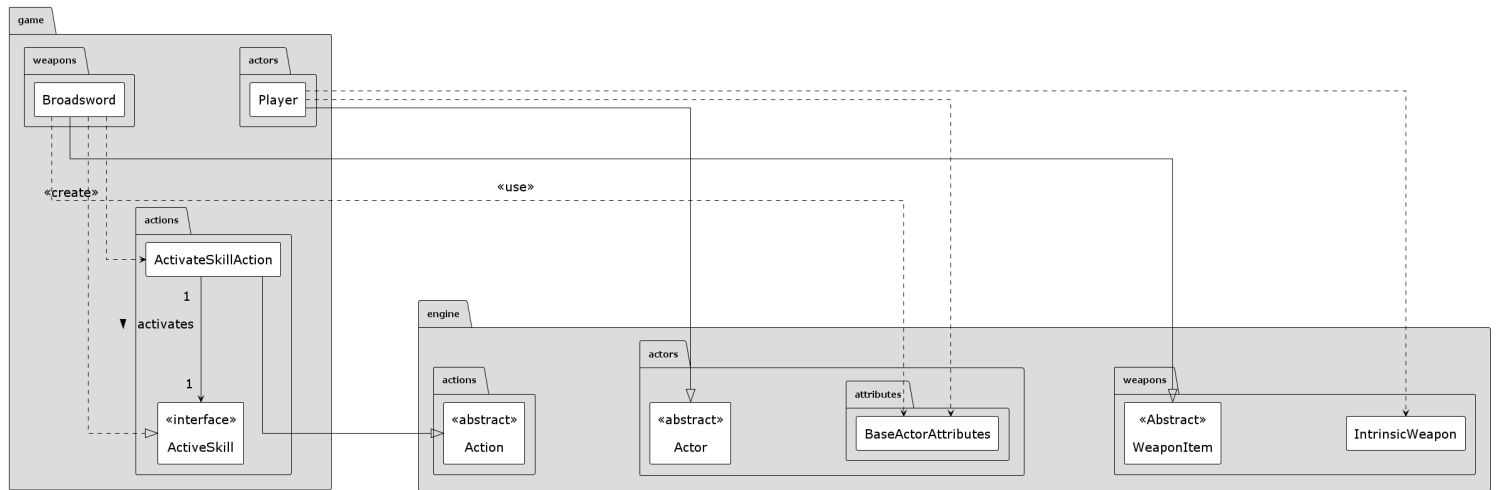


chat.txt

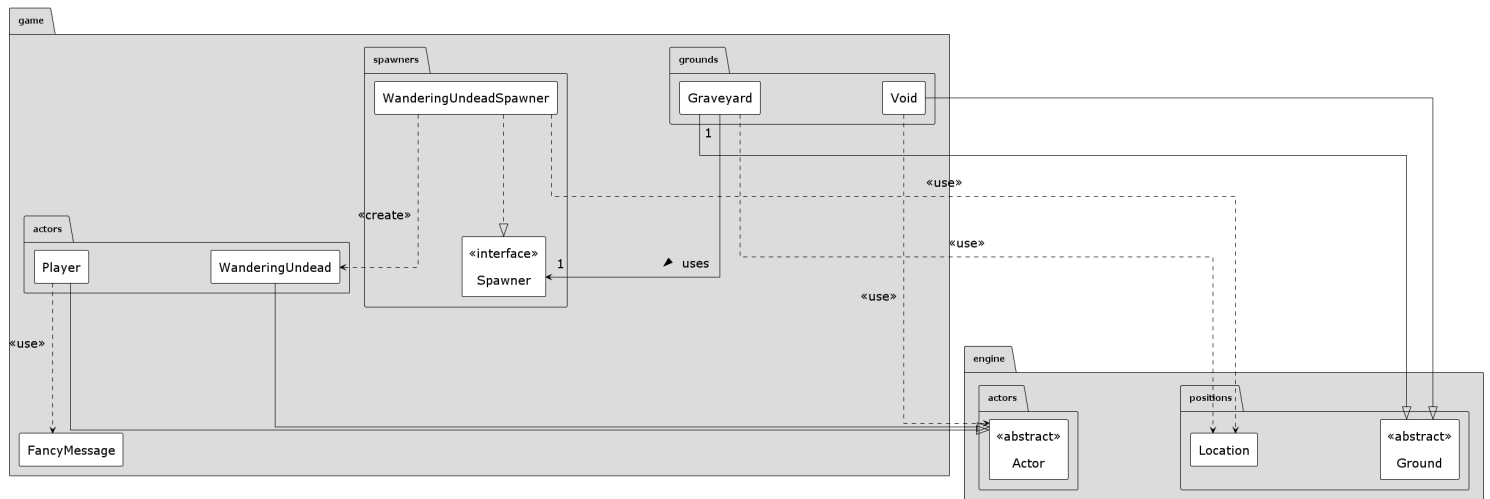


Assignment 1 Sample UML Class Diagrams

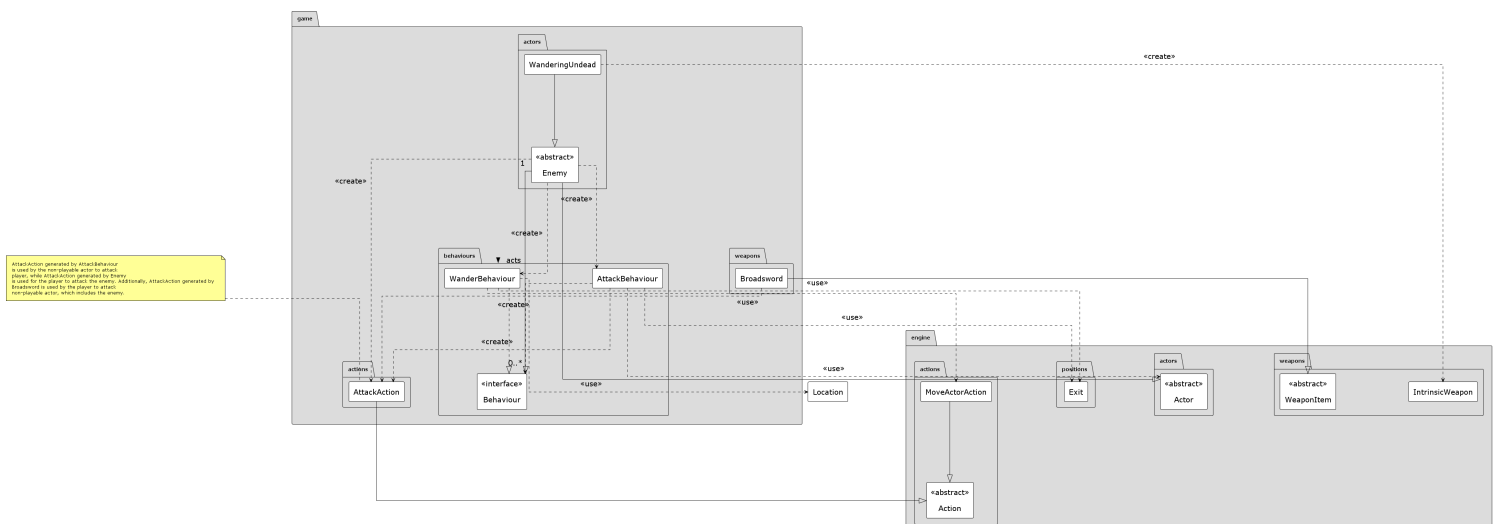
REQ1



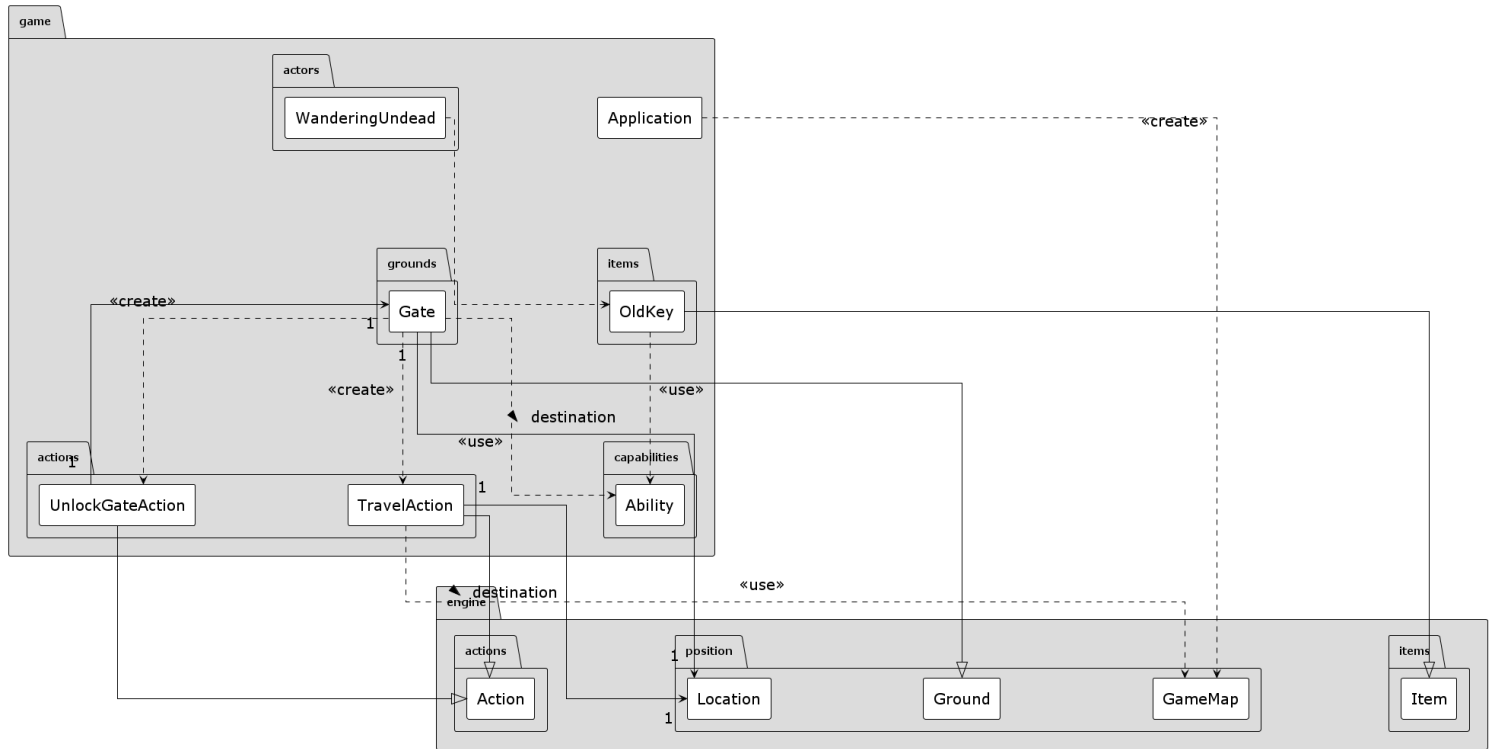
REQ2



REQ3



REQ4



REQ5

